

推理八股文

Prefill/Decode · KV Cache · vLLM 调度 · 量化 · P/D 分离

面向 LLM Serving 与推理引擎面试。回答每题时，先说明请求路径和瓶颈，再说明优化手段改变了什么指标，以及什么负载下不值得开启。

原创整理 · 公开题型检索与官方资料交叉核验 · 核验日期：2026-09-20
社区资料用于发现高频题型；技术结论以文末官方文档与原始论文为准。

建议口述顺序：先给结论，再讲机制与数据流，最后说明性能代价、适用边界和怎样用指标验证。遇到场景题，先澄清负载、硬件、SLO 与约束。

一、请求路径与缓存

Q01 一次流式 LLM 请求会经过哪些阶段？

30 秒回答：典型路径是解析/鉴权与 tokenization → admission/scheduler → prefill → 分配并写入 KV Cache → decode 循环、采样与流式返回 → EOS、取消或超时后释放 block。要能指出队列、cache manager、GPU worker 与指标采集的边界。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q02 Prefill 与 Decode 的本质区别是什么？

30 秒回答：Prefill 一次处理整段 prompt，GEMM 并行度大，常偏计算密集；decode 每轮只追加少量 token，却需读模型权重和历史 KV，常偏访存/调度受限。结论依赖 batch、上下文、模型和硬件，不能绝对化。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q03 KV Cache 如何估算，GQA/MQA 有什么影响？

30 秒回答：标准估算为 $2 \times \text{layers} \times \text{tokens} \times \text{kv_heads} \times \text{head_dim} \times \text{dtype_bytes}$ ，再乘并发序列；还应留出 block 对齐、元数据和临时 buffer。GQA/MQA 减少 kv_heads，因此降低长上下文和 decode 的 KV 带宽与容量压力。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q04 PagedAttention 解决什么问题，没解决什么问题？

30 秒回答：它用固定大小 KV block 和逻辑 block table 映射非连续物理块，降低动态分配碎片并支持共享。它提升有效容量和调度灵活性，但不会减少每个 token 理论上需要的 K/V 数据量，也不能替代显存预算。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q05 Prefix Caching 什么时候有效，什么情况下无效？

30 秒回答：新请求共享完全相同 token 前缀时可复用已算出的 KV block，从而减少 prefill；输出很长、decode 主导或前缀不共享时收益有限。cache key 必须纳入模型/LoRA/多模态输入等影响语义的信息，并考虑多租户隔离。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

二、调度与 SLO

Q06 为什么 online serving 更适合 continuous batching？

30 秒回答：每个 decode iteration 都能插入新请求、移除完成请求，避免静态 batch 等待最慢请求和大量 padding。代价是 scheduler 必须在吞吐、短请求公平性、TTFT 与 P99 之间做显式取舍。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q07 Chunked Prefill 有何价值和代价？

30 秒回答：它把超长 prompt 分块，使 prefill 与 decode 在同一轮 token 预算内共存，保护其他请求的 TPOT。代价是长请求自身 TTFT 可能增加；若 admission 和 KV 预算不当，仍会触发 preemption 或 cache thrashing。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q08 如何设计 admission control？

30 秒回答：按 token 而非只按请求数预算，结合 prompt 长度、最大生成长度、可用 KV blocks、GPU memory 与目标 SLO；可使用 FCFS、优先级或公平队列。过载时要明确拒绝/排队/降级策略，而不是无限堆积队列。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q09 TTFT、ITL、TPOT、E2E、吞吐如何一起解读？

30 秒回答：TTFT 看首次响应，ITL 是流式 token 间隔，TPOT 是后续 token 平均耗时，E2E 包含排队和完整生成，吞吐是系统总产能。比较实验必须带输入/输出长度、并发/到达率、成功率和 P50/P95/P99。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q10 P99 变差时如何系统定位?

30 秒回答: 按请求分解 tokenization、queue、prefill、decode 和网络回传; 同时看 running/waiting 请求、KV 使用、prefix hit、preemption、慢 worker 和错误。用真实 trace 压测复现后, 再针对长 prompt、缓存压力或排队做单变量改动。

追问提示: 补充具体负载、关键指标、失败模式, 以及你会用什么 trace / profiler / 对照实验验证。

三、并行、量化与模型结构

Q11 推理中 DP、TP、PP、EP 如何分工?

30 秒回答: DP 以多个完整 replica 扩容量; TP 切单层张量以容纳模型或加速大 GEMM, 但每层有 collective; PP 切层以分摊权重显存, 会引入 per-request 延迟和气泡; EP 支撑 MoE 专家。应从模型是否单卡可容、网络拓扑、并发和 SLO 做选择。

追问提示: 补充具体负载、关键指标、失败模式, 以及你会用什么 trace / profiler / 对照实验验证。

Q12 P/D 分离为什么可能有用?

30 秒回答: prefill 和 decode 的计算形态与最优并行度不同, 拆到不同 GPU 池可分别扩缩并降低互相干扰。代价是 KV transfer、路由、故障处理和容量预测更复杂; 高并发且同时约束 TTFT/TPOT 时更可能划算。

追问提示: 补充具体负载、关键指标、失败模式, 以及你会用什么 trace / profiler / 对照实验验证。

Q13 权重量化与 KV Cache 量化有什么不同?

30 秒回答: 权重量化减少加载模型和每 token 读取权重的显存/带宽; KV 量化主要释放长上下文、高并发缓存容量。两者都需要验证 scale、group size、dequant kernel、数值回归和是否真的命中当前瓶颈。

追问提示: 补充具体负载、关键指标、失败模式, 以及你会用什么 trace / profiler / 对照实验验证。

Q14 FlashAttention 与 PagedAttention 的关系是什么?

30 秒回答: FlashAttention 是 IO-aware 的注意力计算方法, 避免 materialize 大 attention matrix; PagedAttention 关注 KV Cache 非连续物理块的管理与访问。二者可在同一服务栈中出现, 不能把一个当作另一个的替代。

追问提示: 补充具体负载、关键指标、失败模式, 以及你会用什么 trace / profiler / 对照实验验证。

Q15 MoE 在线推理的尾延迟主要来自哪里？

30 秒回答：Top-k 专家虽减少激活计算，但 expert dispatch/combine 的 AlltoAll、跨节点带宽、专家热度和 grouped GEMM 都会放大尾延迟。应分别看 router 分布、每 expert token 数、A2A 时间、EP placement 与 GEMM shape。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

四、进阶优化与排障

Q16 投机解码为什么可能加速，又为什么可能不加速？

30 秒回答：draft model 或多 token predictor 先提出候选，target model 并行验证；接受率高且草稿开销低时，减少 target 串行 decode 次数。收益取决于接受率、草稿长度、batch、额外 KV/通信和流量饱和度，必须端到端实测。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q17 正确的投机解码会改变输出分布吗？

30 秒回答：满足对应 rejection-sampling 条件的算法可以保持 target 分布；工程实现仍可能受浮点、batch shape 与非确定性影响，未必 bitwise 一致。不能把“使用草稿模型”与“随意贪心接受候选”混为一谈。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q18 CUDA Graph 为什么对 decode 有帮助？

30 秒回答：decode 常重复执行短 kernel 序列，capture 后 replay 可减少 CPU/driver launch 开销与 GPU 空档。动态 shape、混合 prefill/decode 和不可 capture 算子需 shape bucket、分模式 graph 或 eager fallback；graph 不是 kernel 内部优化的替代品。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q19 KV Cache 快满时的正确处理顺序？

30 秒回答：先检查实际 context/并发/最大生成长度是否超出容量模型，再通过 admission、max model len、max batched tokens 和保留空间约束避免过度准入；随后评估 cache 量化、更多 replica、扩卡或 offload。仅增大 GPU memory utilization 可能让临时 buffer 不足。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q20 如何做可复现的推理 benchmark?

30 秒回答: 固定模型、版本、GPU、驱动、精度和 sampling; 预热后使用真实或代表性的输入/输出长度、并发与 arrival pattern, 记录 TTFT/TPOT/E2E 的 percentile、tokens/s、错误率和显存。区分 offline throughput、online serving 与端到端 API 三类测试。

追问提示: 补充具体负载、关键指标、失败模式, 以及你会用什么 trace / profiler / 对照实验验证。

@小冯别放弃

60 秒答题模板

1. 定义与目标：先说它解决哪一个资源、正确性或 SLO 问题。
2. 机制与数据流：指出数据/张量如何移动，计算和通信在哪里发生。
3. 取舍：说明换来了什么显存、带宽、计算、复杂度或尾延迟代价。
4. 验证：给出应看的指标、对照实验和最小复现方式。

@小冯别放弃

资料来源与使用边界

以下社区面经/题库用于归纳问题主题；答案为原创整理。框架和硬件会演进，实际配置与行为应以对应版本的官方文档、源码和基准测试为准。

- 牛客: C++/CUDA/AI Infra 面试经验总结
- 牛客: 2025 春招 AI Infra 面经汇总
- GitHub: AI Infra 核心题清单
- GitHub: AI Infra 学习与面试高频问题
- GitHub: LLM/RL Infra 面试问题整理
- vLLM: Automatic Prefix Caching
- vLLM: Chunked Prefill 与 Scheduler 参数
- vLLM: Disaggregated Prefill
- PagedAttention 论文
- Speculative Decoding 论文
- DistServe: P/D 分离论文

@小冯别放弃